The `TemplateDispatcher<>` class template is modeled on the `dispatcher` class and is almost identical. In particular, the destructor still calls `wait_and_dispatch()` to wait for a message.

Since you don't throw exceptions if you handle the message, you now need to check whether you did handle the message in your message loop ❶. Your message processing stops when you've successfully handled a message, so that you can wait for a different set of messages next time. If you do get a match for the specified message type, the supplied function is called ❷ rather than throwing an exception (although the handler function may throw an exception itself). If you don't get a match, you chain to the previous dispatcher ❸. In the first instance, this will be a `dispatcher`, but if you chain calls to `handle()` ❹ to allow multiple types of messages to be handled, this may be a prior instantiation of `TemplateDispatcher<>`, which will in turn chain to the previous handler if the message doesn't match. Because any of the handlers might throw an exception (including the `dispatcher`'s default handler for `close_queue` messages), the destructor must once again be declared `noexcept(false)` ❺.

This simple framework allows you to push any type of message on the queue and then selectively match against messages you can handle on the receiving end. It also allows you to pass around a reference to the queue for pushing messages on, while keeping the receiving end private.

To complete the example from chapter 4, the messages are given in listing C.6, the various state machines in listings C.7, C.8, and C.9, and the driving code in listing C.10.

Listing C.6. ATM messages

```
struct withdraw
{
    std::string account;
     unsigned amount;
     mutable messaging::sender atm_queue;

     withdraw(std::string const& account_,
              unsigned amount_,
```